
Projets AI & Big Data

Les sujets a maitriser : du pipeline de donnees au deploiement

TCHAYE-KONDI Jude, Ph.D.

Cours de 28 heures — Suivi de projet

Plan du cours

- 1 Un projet = un systeme complet
- 2 Topic 1 — Donnees & Big Data
- 3 Topic 2 — Les familles de modeles
- 4 Topic 3 — Evaluation
- 5 Topic 4 — API & Application
- 6 Topic 5 — Deploiement
- 7 Methode & livrables
- 8 Synthese

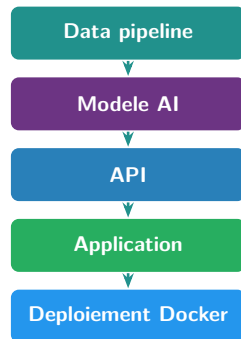
Un projet = un systeme complet

Ce qu'on attend de vous

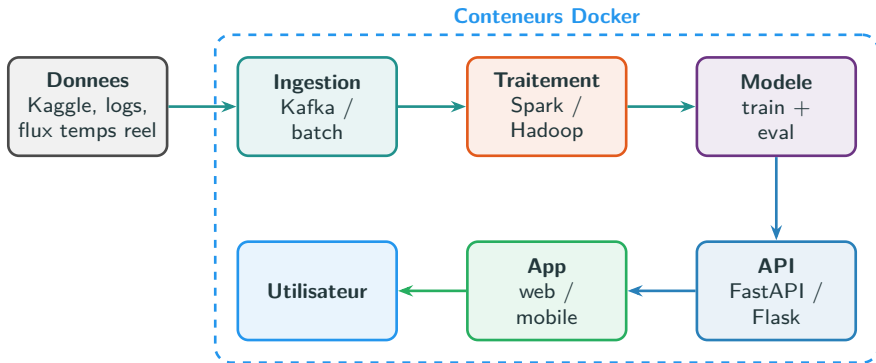
Un projet n'est **pas** un notebook avec un modele. C'est un **systeme complet** de bout en bout :

- un **pipeline de donnees** (batch ou streaming),
- un **modele AI** entraine et evalue,
- une **API** qui sert le modele,
- une **application** (web / mobile / desktop),
- un **deploiement** reproductible (Docker).

Chaque brique correspond a un **role** dans l'equipe.



L'architecture cible (vue d'ensemble)



Intuition

Quel que soit le sujet (fraude, recommandation, vision, NLP...), c'est *toujours* ce squelette.

Seules les briques changent.

Les roles dans l'equipe

Role	Responsabilite	Briques
Data Engineer	Ingestion, nettoyage, pipeline	Spark, Kafka, Hadoop
ML Engineer	Modele, entraînement, evaluation	PyTorch, MLlib, XGBoost
Backend Dev	API, integration, donnees servies	FastAPI, Flask
Frontend Dev	Application, visualisation	Streamlit, React

A retenir

Equipe de 2 a 3 personnes. Chaque membre a un role **clairement defini** et des contributions **precises** a documenter dans le rapport (sinon : penalite).

Topic 1 — Donnees & Big Data

La donnée avant tout

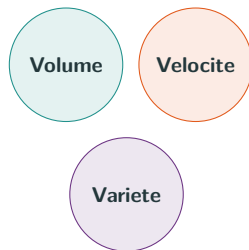
Avant tout modèle, il faut **comprendre et préparer** la donnée.

- **Source** : Kaggle, open data, flux temps réel.
- **Volume** : tient-il en RAM ? Sinon → Big Data.
- **Qualité** : valeurs manquantes, doublons, bruit.
- **Biais** : classes déséquilibrées (fraude $\sim 0,2\%$).

Attention

80% du temps d'un projet, c'est la donnée.
Sous-estimer cette phase = projet bloqué.

Les 3 V du Big Data



Quand un seul PC ne suffit plus.

Batch vs Streaming

Batch (par lots)

- On traite un gros volume *deja stocke*.
- Tolerant a la latence (minutes, heures).
- Ex : recommandation, scoring credit, data lake.



Streaming (temps reel)

- On traite la donnee *a la volee*, en continu.
- Faible latence (secondes).
- Ex : fraude live, trafic, churn, monitoring.



Intuition

Batch = *photo* du passe. Streaming = *video* du present. Votre sujet impose souvent l'un des deux.

L'écosystème Big Data

Outil	Role	Quand l'utiliser
Spark	Calcul distribue en memoire	Traitement massif, ML a l'echelle
Hadoop/HDFS	Stockage distribue	Data lake, gros fichiers
Hive	SQL sur Big Data	Requetes analytiques
Kafka	Bus de messages temps reel	Ingestion de flux

Conseil encadrant

Ne sortez pas l'artillerie lourde inutilement. Spark sur 10 000 lignes est *plus lent* que pandas. Justifiez le choix Big Data par le **volume reel**.

EDA : explorer avant de modeliser

L'analyse exploratoire (**EDA**) répond à : *a quoi ressemble ma donnée ?*

```
df.describe()           # stats : moyenne, min, max, quartiles
df.isna().mean()        # % de valeurs manquantes par colonne
df["classe"].value_counts(normalize=True)  # équilibre des classes
df.corr()                # corrélations entre variables
```

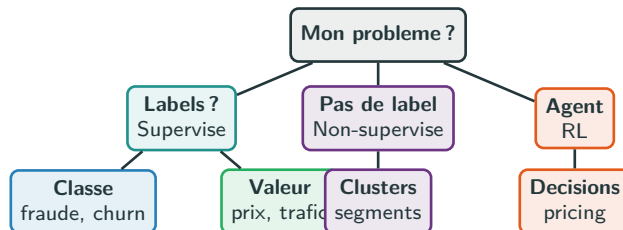
- Distributions, valeurs aberrantes, corrélations.
- Détecter les **biais** et le **déséquilibre** tot.
- Décider le **feature engineering** à faire.

A retenir

Une bonne EDA = la moitié du travail de modélisation. À inclure dans le rapport (section 2).

Topic 2 — Les familles de modeles

Quel type de probleme ? Quel modele ?



Intuition

Le **type de probleme** (classification, regression, clustering, RL...) dicte la famille de modeles. C'est la 1^{re} question a se poser.

Apprentissage supervise (SL)

On a des **exemples etiquetes** (x, y) : le modele apprend $x \rightarrow y$.

Tache	Sortie	Modeles	Sujets
Classification	classe (0/1, A/B/C)	XGBoost, RandomForest, BERT	fraude, fake news, churn
Regression	valeur continue	LSTM, regression, GBM	trafic, prix, RUL panne

Exemple

Scoring credit (sujet 15) : x = profil client, y = default/pas default. XGBoost + explicabilite (SHAP) pour justifier la decision.

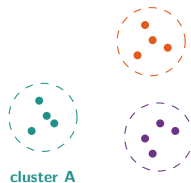
Attention

Eviter le **data leakage** : ne jamais utiliser une info du futur (ou la cible deguisee) comme feature.

Apprentissage non-supervise

Pas de label : on cherche une **structure cachee**.

- **Clustering** (KMeans, DBSCAN) : regrouper des points similaires → segmentation client.
- **Reduction de dimension** (PCA, autoencoder) : compresser l'information.
- **Detection d'anomalies** : ce qui sort de la norme → fraude, pannes.



KMeans regroupe par proximite.

Conseil encadrant

Pour le clustering : toujours **interpreter** les clusters (qui sont ces clients?). Un cluster non explique ne vaut rien.

Deep Learning : quand et pourquoi

Le DL brille sur les donnees **non structurees** : images, texte, sequences.

- **CNN / U-Net** : images (vision, segmentation agricole).
- **LSTM** : series temporelles (trafic, pannes, finance).
- **Transformers / BERT** : texte (fake news, sentiment, RAG).
- **Autoencoder** : compression, anomalies.



Reseau de neurones : entree → couches cachees → sortie.

Attention

Le DL n'est pas toujours la reponse. Sur du tabulaire, **XGBoost bat souvent un reseau** et s'entraîne en secondes.

NLP, Vision & cas particuliers

Domaine	Techniques	Sujets concernés
NLP	BERT, embeddings, FAISS, RAG	fake news, sentiment, recherche, chatbot
Vision	YOLO, CNN, U-Net	detection objets, agriculture
Series temp.	LSTM, ARIMA	trafic, finance, pannes
Graphes	GraphX, Graph ML	bots reseaux sociaux
RL	DQN, policy gradient	pricing dynamique, trading

Intuition

Recherche semantique & RAG : on transforme le texte en **vecteurs** (embeddings), puis on cherche par **sens** (distance entre vecteurs) plutot que par mot-cle.

Topic 3 — Evaluation

Mesurer, c'est progresser

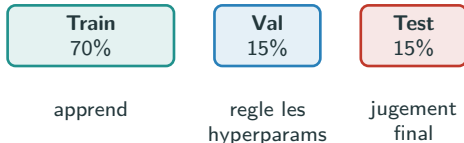
Un modele sans **metrique** n'est pas evaluable. La metrique depend de la tache.

Tache	Metriques	Attention
Classification	Accuracy, F1 , AUC, precision/rappel	accuracy trompeuse si desequilibre
Regression	RMSE, MAE, R^2	sensible aux outliers
Clustering	Silhouette, inertie	+ interpretation humaine
Detection objets	mAP, IoU	seuil de confiance

Attention

Fraude : 99,8% des transactions sont normales. Un modele qui dit *"jamais de fraude"* a 99,8% d'accuracy... et 0% d'utilite. **Utilisez le F1 / rappel.**

Methodologie rigoureuse



- Toujours comparer a une **baseline** simple.
- Comparer **plusieurs modeles** (tableau dans le rapport).
- Analyser les **erreurs** : ou et pourquoi le modele se trompe.

A retenir

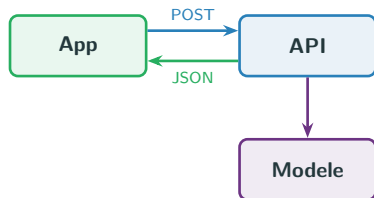
Le jeu de **test** ne sert qu'**une seule fois**, a la toute fin. Sinon vous trichez sans le savoir.

Topic 4 — API & Application

Servir le modele : l'API

L'API expose le modele au monde exterieur via HTTP.

- **FastAPI** ou **Flask**.
- Un endpoint `/predict` : entree $\rightarrow$ prediction.
- Validation des entrees, codes HTTP, JSON.
- Charge le modele *une fois* au demarrage.



```
@app.post("/predict")
def predict(features: Transaction):
    score = model.predict_proba([features.to_array()])[0][1]
    return {"fraude": score > 0.5, "score": float(score)}
```

L'application : rendre le systeme utilisable

Le modele seul **ne suffit pas** (regle : modele sans app = REFUSE). Il faut une interface.

Type	Outils	Adapte a
Web	Streamlit, React, Flask	dashboards, checkers, la plupart des sujets
Mobile	Flutter, Android	app photo (agriculture), terrain
Desktop	PyQt, Electron	outils internes

L'application doit permettre de : **tester** le modele, **visualiser** les resultats, **interagir** avec le systeme.

Conseil encadrant

Streamlit est le chemin le plus rapide pour un dashboard data en Python. Ideal quand le frontend n'est pas votre specialite.

Topic 5 — Deploiement

Docker : "ca marche sur ma machine"... partout

Docker **empaquette** votre app + ses dependances dans un **conteneur** reproductible : plus de "ca marche chez moi", meme environnement partout, une commande pour tout lancer.

```
# docker-compose.yml  -- orchestre tous les services
services:
  api:
    build: ./api
    ports: ["8000:8000"]
  app:
    build: ./app
    ports: ["8501:8501"]
    depends_on: [api]
  kafka:
    image: bitnami/kafka
```

Intuition

Un conteneur = une boite scellee avec l'app, Python, les libs, la config. On l'ouvre n'importe ou, ca tourne pareil.

Du code au systeme deployable

- 1 **Dockerfile** par service (API, app).
- 2 **docker-compose** pour orchestrer le tout (API + app + Kafka + DB).
- 3 Variables d'environnement, ports, volumes.
- 4 **README** : instructions d'installation et d'execution.

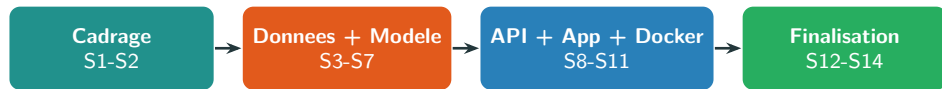
```
# Tout le systeme démarre en une commande :  
docker compose up --build
```

A retenir

Le correcteur doit pouvoir **cloner votre repo et lancer une commande**. S'il faut 2h de configuration manuelle, c'est rate.

Methode & livrables

Le fil rouge sur 28 heures



- **14 seances de 2h.** Feedback a chaque seance.
- Chaque seance produit un **livrable concret**.
- Voir le document *"Calendrier de suivi"* pour le detail.

Conseil encadrant

Avancez par **tranches verticales** : un mini bout de bout-en-bout qui marche tot, puis on ameliore. Pas tout le modele d'abord, l'app a la fin.

Les livrables obligatoires

1. Rapport PDF

- 5 a 20 pages.
- Schema d'architecture obligatoire.
- Repartition du travail detaillee.
- Envoye a tchaye59@gmail.com.

2. Code GitHub

- Repo propre et documente.
- README : install + execution.

3. Application fonctionnelle

- Web / mobile / desktop.
- Tester, visualiser, interagir.

4. Presentation

- 15 a 20 minutes.
- Demonstration live.
- Architecture, choix, resultats.

Attention

Plagiat = 0. Copie d'un projet Kaggle sans adaptation = 0. Modele seul sans application = REFUSE.

Synthese

Ce qu'il faut retenir

A retenir

- Un projet = un **systeme complet** : donnees $\rightarrow$ modele $\rightarrow$ API $\rightarrow$ app $\rightarrow$ Docker.
- La **donnee** (EDA, qualite, biais) conditionne tout le reste.
- Le **type de probleme** dicte la famille de modeles (SL, non-supervise, DL, RL...).
- **Evaluez** avec la bonne metrique ; attention au desequilibre des classes.
- L'**API** sert le modele, l'**application** le rend utilisable.
- **Docker** rend le tout reproductible et deployable en une commande.
- Travaillez en **equipe**, avec des roles clairs et un suivi a chaque seance.

Au travail.

Choisissez un sujet, formez votre equipe, et construisez un vrai systeme.